

# UART(tx)

## **Universal Asynchronous Receiver / Transmitter** RTL Design, Testbench Development & Functional Verification Report

**Name/ Mahmoud Shawky**

**S26-B8-FPGA-G8**

# Objective\_tx

**Design a configurable UART Transmitter.**

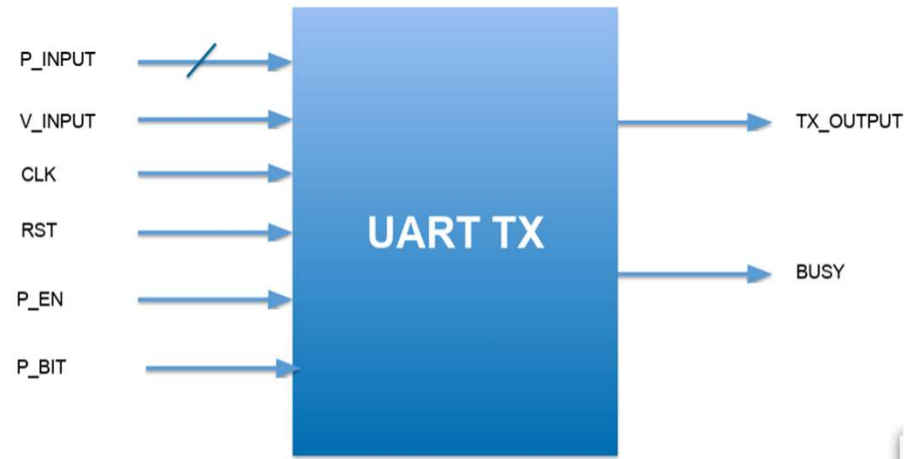
**Transmit 8-bit parallel data serially.**

**Support optional parity bit.**

**Support Even and Odd parity.**

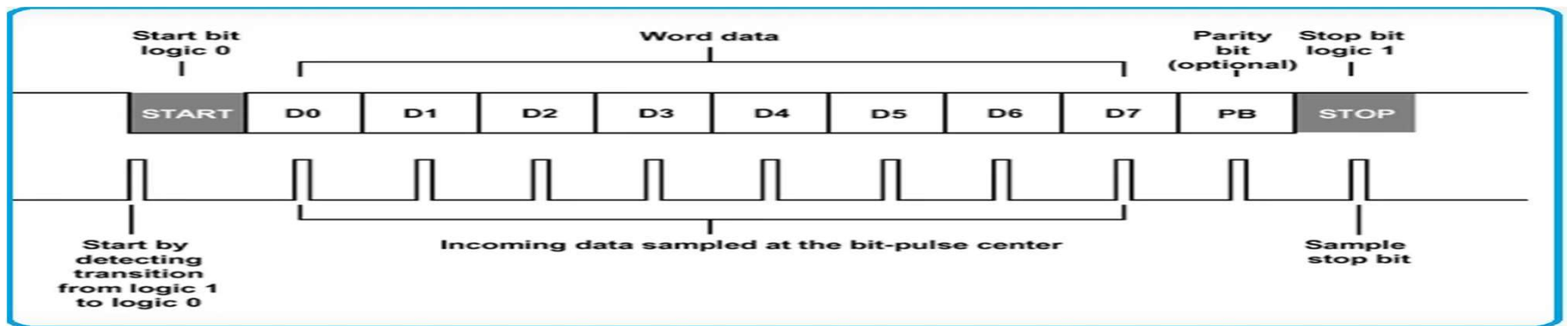
**Generate Start and Stop bits.**

**Control transmission using an FSM.**

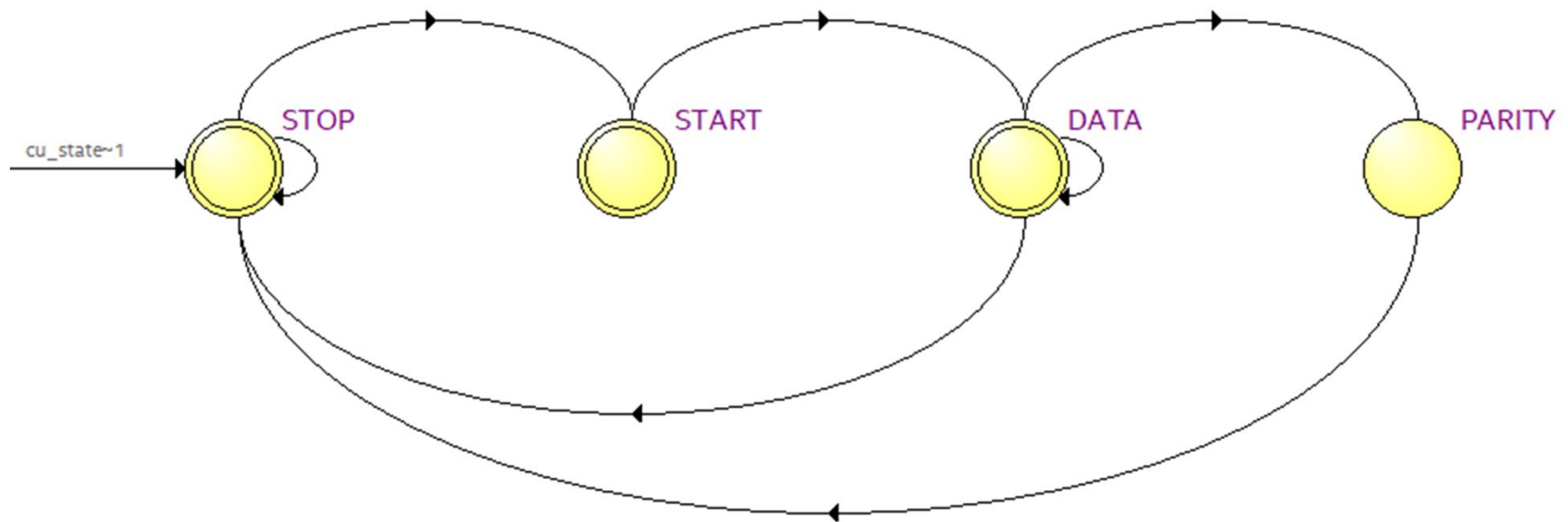


# uart\_tx — Required Behaviour

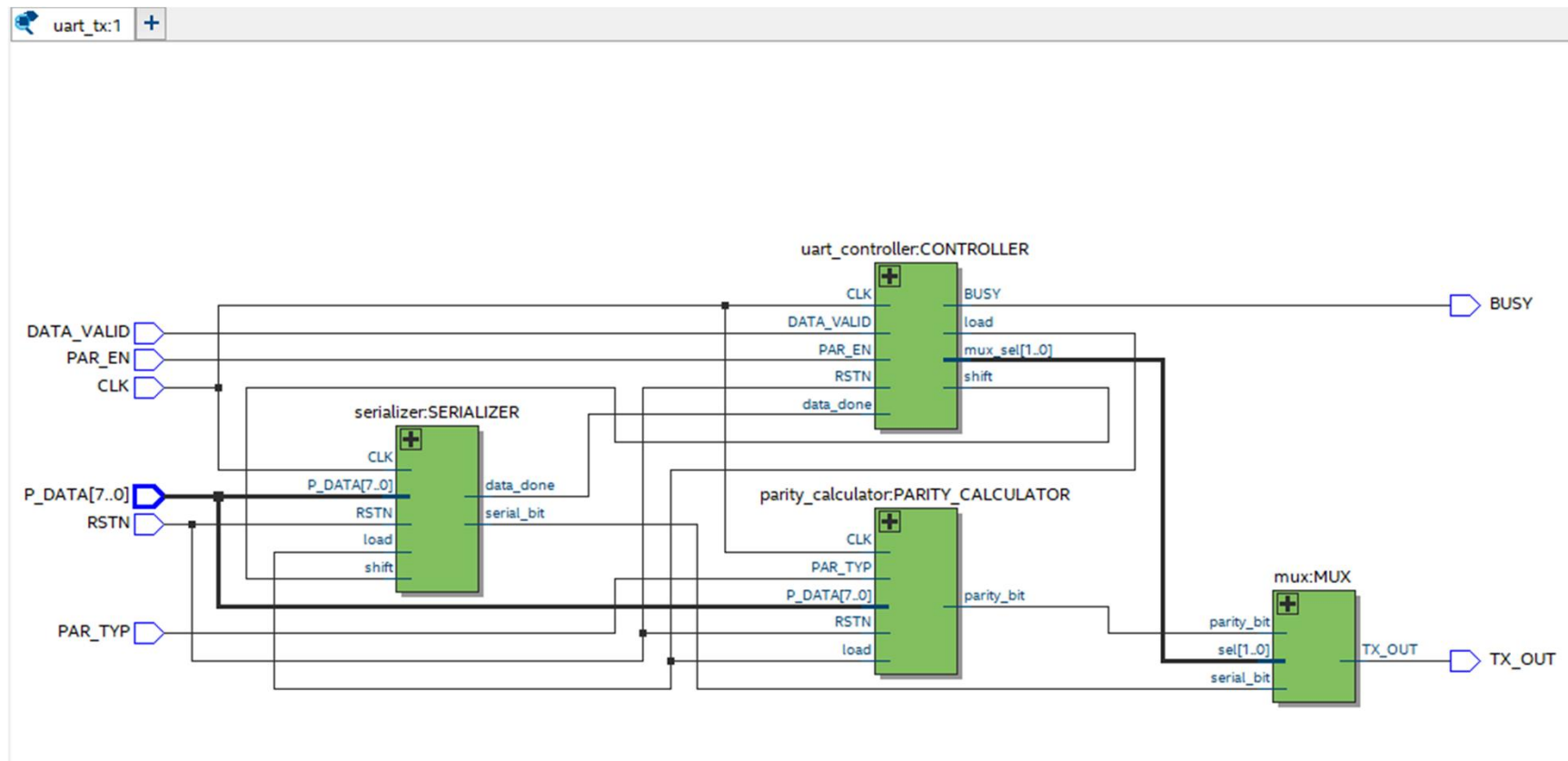
- Out of reset: o\_tx = 1, o\_busy = 0
- A byte is accepted only when i\_valid = 1 AND o\_busy = 0 — same cycle, both true
- i\_valid while o\_busy = 1 is ignored — no queuing
- After acceptance, o\_tx carries the frame one bit per cycle, then returns to idle-high
- o\_busy = 1 for exactly the frame duration (10 or 11 cycles), 0 otherwise
- Changing i\_data after the accept cycle must not affect the frame already in flight



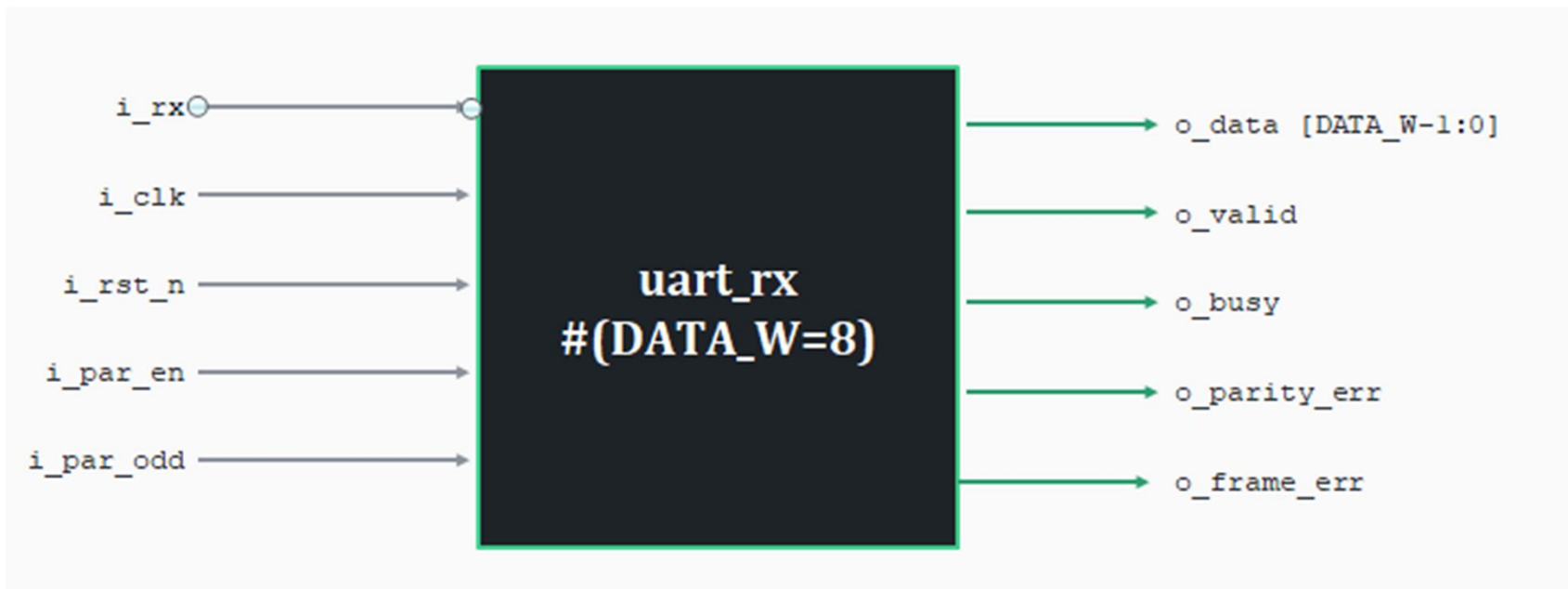
# FSM(Controller )



# System Architecture\_tx

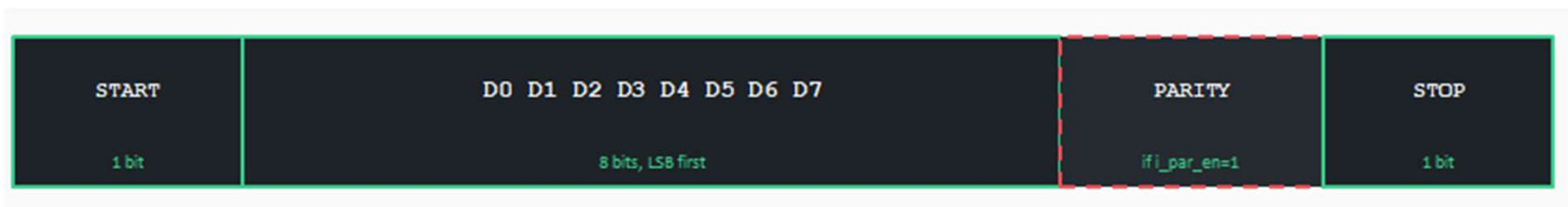


# Objective\_rx

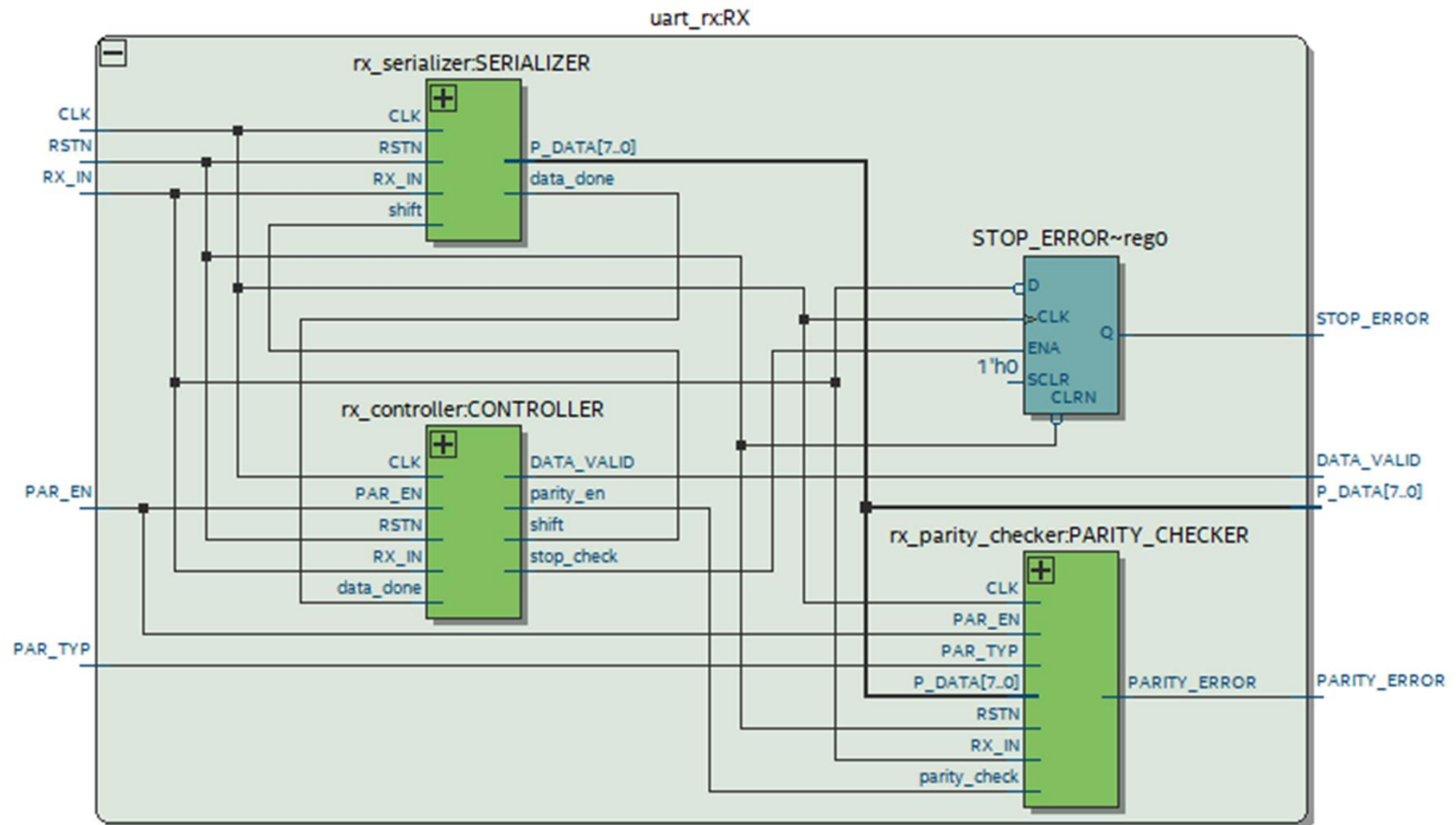


# uart\_rx — Required Behaviour

- Out of reset: o\_busy = 0, outputs low
- Detects the start bit only while idle — a 0 inside the data byte is never a new start
- After a full frame, o\_valid pulses for exactly 1 cycle with o\_data holding the received byte
- Flag-and-deliver: even on error, o\_valid still pulses and o\_data still carries the byte
- o\_frame\_err pulses (same cycle as o\_valid) when the stop-bit position isn't 1
- o\_parity\_err pulses when i\_par\_en=1 and received parity disagrees — always 0 when i\_par\_en=0

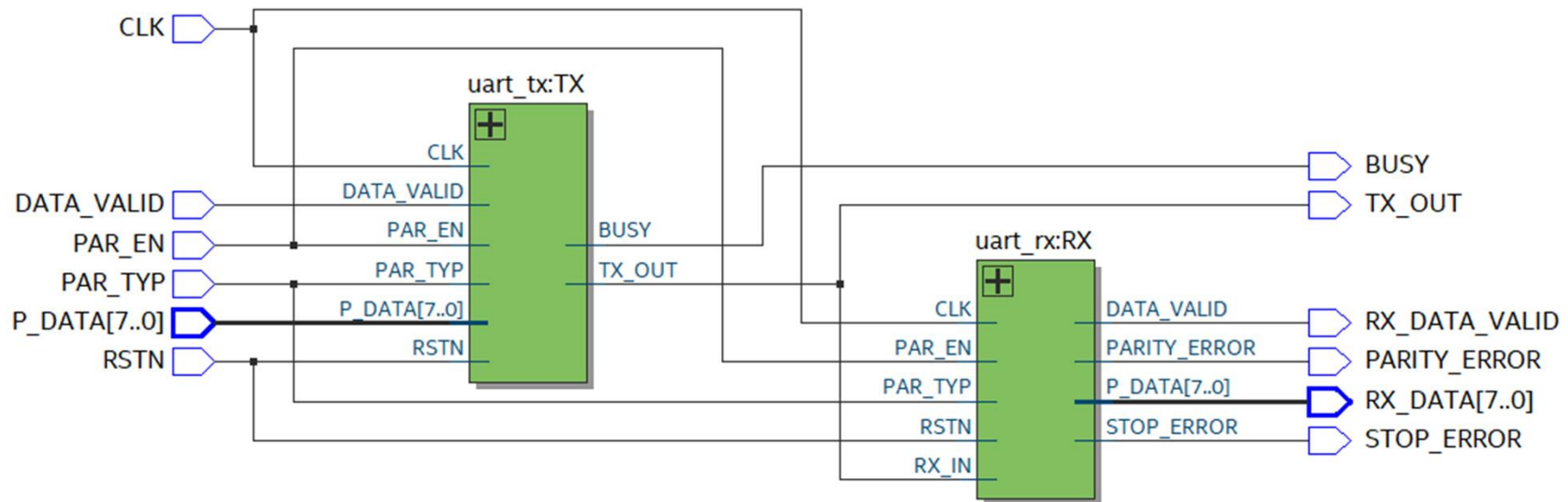


# System Architecture \_rx





# Uart(full)



**RX and TX communicate through shared timing (same clock) and a pre-agreed frame format (START, DATA, PARITY, STOP). TX places bits on the wire one by one, and RX reads them at the exact same times using its own FSM and counters. RX then validates the data using parity and stop bit checking. RX sends no direct feedback to TX; both operate independently based on the clock and the protocol agreement**

# How RX Talks to TX – UART Communication

## - Pre-Communication Agreement (The Handshake Before Starting)

For RX to understand TX, both must agree on two things beforehand:

### 1 Baud Rate (Speed)

Both operate on the same clock signal.

Each bit takes **one clock cycle** to transmit/receive.

### 2 Frame Format (Structure)

Both agree that the frame consists of: → START | DATA (8 bits) | [PARITY] | STOP

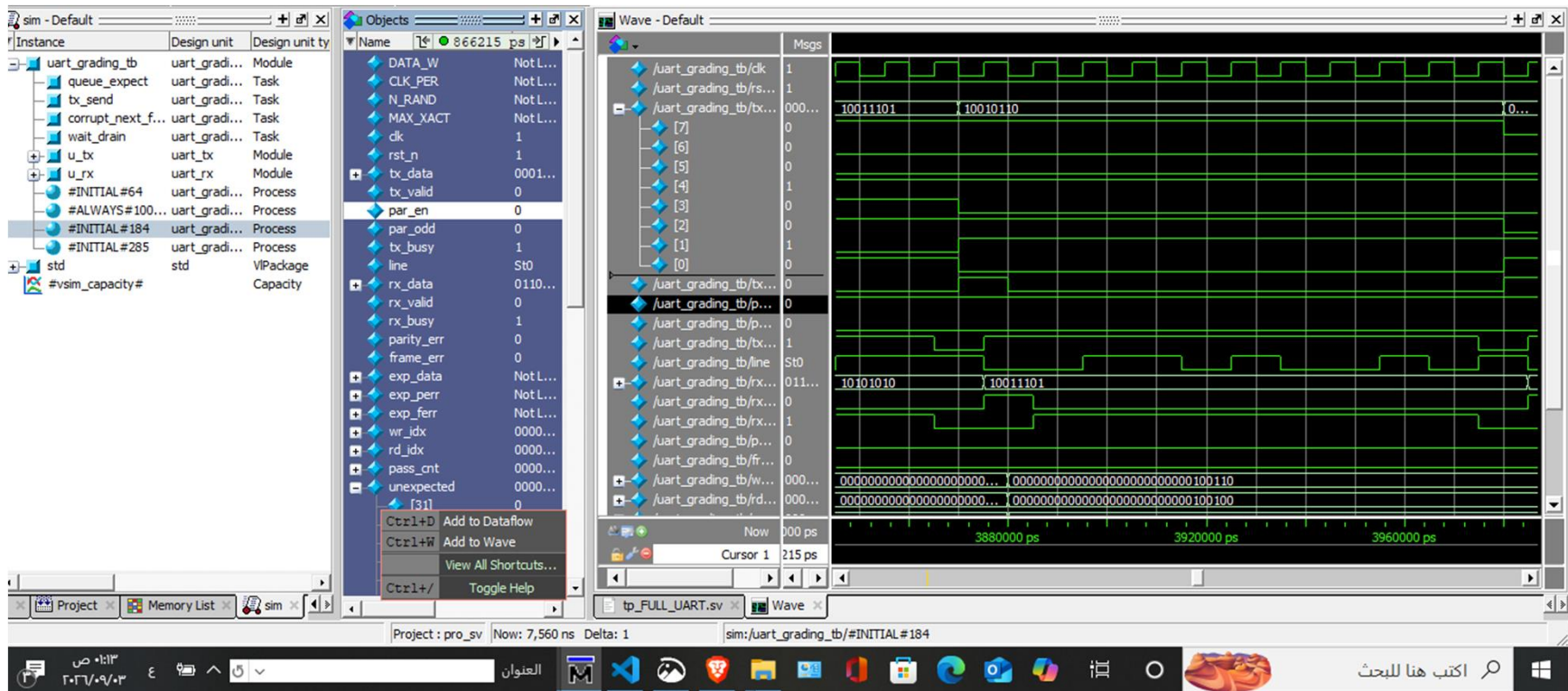
---

Only **TX\_OUT ↔ RX\_IN** is the direct connection.

All other signals are **status/error reports** to the outside world, **not** feedback to the other side.

TX does not know if RX received correctly; RX simply listens, decodes, and reports errors.

# Wave form



# Output

```
# ok[38]: d=0x0a pe=0 fe=0
# ok[39]: d=0x53 pe=0 fe=0
# === 3) random data, P_EN=1 P_BIT=1 (odd) ===
# ok[40]: d=0x6b pe=0 fe=0
# ok[41]: d=0xd5 pe=0 fe=0
# ok[42]: d=0x02 pe=0 fe=0
# ok[43]: d=0xae pe=0 fe=0
# ok[44]: d=0x1d pe=0 fe=0
# ok[45]: d=0xcf pe=0 fe=0
# ok[46]: d=0x23 pe=0 fe=0
# ok[47]: d=0x0a pe=0 fe=0
# ok[48]: d=0xca pe=0 fe=0
# ok[49]: d=0x3c pe=0 fe=0
# ok[50]: d=0xf2 pe=0 fe=0
# ok[51]: d=0x8a pe=0 fe=0
# ok[52]: d=0x41 pe=0 fe=0
# ok[53]: d=0xd8 pe=0 fe=0
# ok[54]: d=0x78 pe=0 fe=0
# ok[55]: d=0x89 pe=0 fe=0
# ok[56]: d=0xeb pe=0 fe=0
# ok[57]: d=0xb6 pe=0 fe=0
# ok[58]: d=0xc6 pe=0 fe=0
# ok[59]: d=0xae pe=0 fe=0
# === 4) injected parity error ===
# ok[60]: d=0xc3 pe=1 fe=0
# === 5) injected framing error ===
# ok[61]: d=0x5c pe=0 fe=1
# === 6) back-to-back frames ===
# ok[62]: d=0xa5 pe=0 fe=0
# ok[63]: d=0x5a pe=0 fe=0
# ok[64]: d=0xff pe=0 fe=0
# === 7) busy-reject ===
# ok[65]: d=0x39 pe=0 fe=0
#
# frames sent      : 66
# frames delivered : 66
# unexpected frames: 0
# SCORE 66 66
# GRADING: 66 / 66 frames correct
# RESULT: PASS
# ** Note: $finish      : D:/mo/project_nti_sv/tp_FULL_UART.sv(279)
```

---